

EX NO : 9

DATE : 29-07-26

Imbalance dataset handling-SMOTE

AIM:

To Consider an imbalance dataset(one that has skewed classes). You may generate one synthetically Check if the data is imbalanced (hint: class count as graph). Apply Synthetic Minority Oversampling Technique to balance it. Build a logistic regression classifier and show the performance before and after applying SMOTE

PROBLEM STATEMENT:

This synthetic dataset simulates 5,000 financial transactions with a highly imbalanced 95:5 ratio, where only 5% represent fraudulent cases.

It contains four numerical features (such as transaction amount and distance from home), blending actual predictive signals with random noise to realistically challenge machine learning models.

CODE:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix
from imblearn.over_sampling import SMOTE

# STEP 1: Generate Synthetic Imbalanced Data
# We simulate a "Fraud Detection" dataset where class 1 (Fraud) is rare.
# weights=[0.95, 0.05] means 95% legitimate transactions, 5% fraud.
X, y = make_classification(
    n_samples=5000,
    n_features=4,
    n_informative=2,
    n_redundant=0,
    n_clusters_per_class=1,
    weights=[0.95, 0.05],
    random_state=42,
    flip_y=0.01
)

# Convert to a DataFrame for easier handling
df = pd.DataFrame(X, columns=['Transaction_Amount', 'Time_of_Day', 'User_Age', 'Distance_from_Home'])
df['Is_Fraud'] = y

# STEP 2: Check for Class Imbalance (Graph)
plt.figure(figsize=(6, 4))
sns.countplot(data=df, x='Is_Fraud', hue='Is_Fraud', palette='Set2', legend=False)
plt.title('Class Distribution Before SMOTE (Imbalanced)')
plt.xlabel('Class (0 = Legitimate, 1 = Fraud)')
plt.ylabel('Count')
plt.show()

print("Class counts before SMOTE:")
print(df['Is_Fraud'].value_counts())
print("-" * 50)

# Split the data into training and testing sets
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)
```

```
# STEP 3: Baseline Logistic Regression
```

```
print("\n--- BASELINE MODEL (Before SMOTE) ---")
```

```
baseline_model = LogisticRegression(random_state=42)
```

```
baseline_model.fit(X_train, y_train)
```

```
y_pred_baseline = baseline_model.predict(X_test)
```

```
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_baseline))
```

```
print("\nClassification Report:\n", classification_report(y_test, y_pred_baseline))
```

```
# STEP 4: Apply SMOTE
```

```
# Apply SMOTE *only* to the training data to prevent data leakage
```

```
smote = SMOTE(random_state=42)
```

```
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)
```

```
# Visualize the balanced training data
```

```
plt.figure(figsize=(6, 4))
```

```
sns.countplot(x=y_train_smote, hue=y_train_smote, palette='Set1', legend=False)
```

```
plt.title('Class Distribution After SMOTE (Balanced Training Data)')
```

```
plt.xlabel('Class (0 = Legitimate, 1 = Fraud)')
```

```
plt.ylabel('Count')
```

```
plt.show()
```

```
print("\nClass counts in Training Set AFTER SMOTE:")
```

```
print(pd.Series(y_train_smote).value_counts())
```

```
print("-" * 50)
```

```
# STEP 5: Train Model on Balanced Data
```

```
print("\n--- SMOTE MODEL (After SMOTE) ---")
```

```
smote_model = LogisticRegression(random_state=42)
```

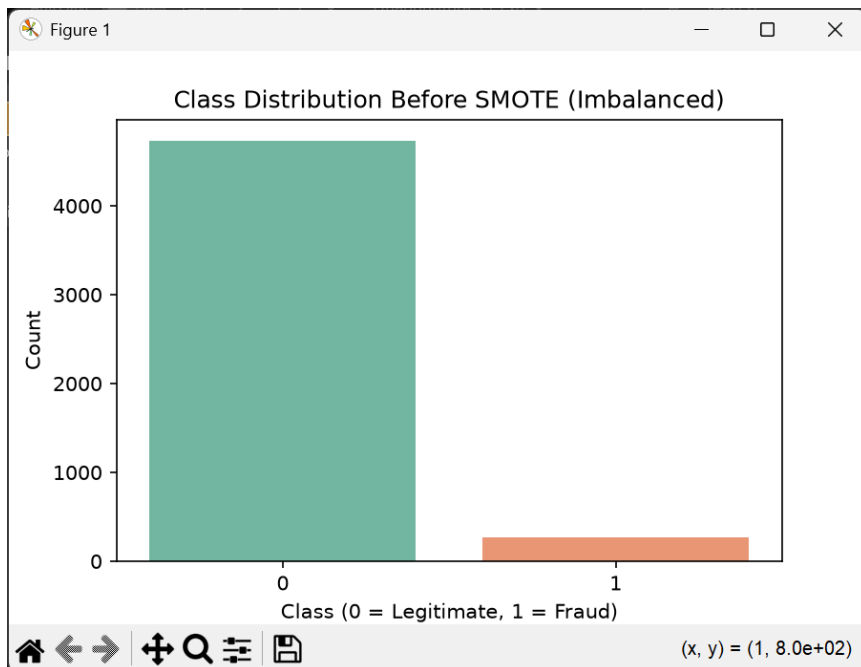
```
smote_model.fit(X_train_smote, y_train_smote)
```

```
y_pred_smote = smote_model.predict(X_test)
```

```
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_smote))
```

```
print("\nClassification Report:\n", classification_report(y_test, y_pred_smote))
```

OUTPUT:





Class counts before SMOTE:

Is_Fraud

0 4732

1 268

Name: count, dtype: int64

--- BASELINE MODEL (Before SMOTE) ---

Confusion Matrix:

[[1407 13]

[42 38]]

Classification Report:

	precision	recall	f1-score	support
0	0.97	0.99	0.98	1420
1	0.75	0.47	0.58	80
accuracy			0.96	1500
macro avg	0.86	0.73	0.78	1500
weighted avg	0.96	0.96	0.96	1500

Class counts in Training Set AFTER SMOTE:

0 3312

1 3312

Name: count, dtype: int64

--- SMOTE MODEL (After SMOTE) ---

Confusion Matrix:

```
[[1263  157]
 [  14   66]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.99	0.89	0.94	1420
1	0.30	0.82	0.44	80
accuracy			0.89	1500
macro avg	0.64	0.86	0.69	1500
weighted avg	0.95	0.89	0.91	1500